

Fase 3 – pruebas de funcionamiento

E3.1 ¿Qué equipos ganan de forma consistente Y con un piso alto?

The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query is as follows:

```
SELECT t.abbreviation AS equipo,
       t.full_name,
       ROUND(AVG(a.pct), 4) AS pct_promedio,
       ROUND(MIN(a.pct), 4) AS piso_peor_temporada,
       ROUND(STDDEV_SAMP(a.pct), 4) AS variabilidad,
       ROUND(AVG(a.margen), 2) AS margen_promedio
FROM analysis_team_season a
JOIN team t ON t.team_id = a.team_id
WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020
GROUP BY t.team_id, t.abbreviation, t.full_name
HAVING COUNT(*) = 6 AND MIN(a.pct) >= 0.400
ORDER BY pct_promedio DESC;
```

The 'Data Output' tab shows the results of the query, displaying 9 rows of data. The columns are: equipo, full_name, pct_promedio, piso_peor_temporada, variabilidad, and margen_promedio.

equipo	full_name	pct_promedio	piso_peor_temporada	variabilidad	margen_promedio
1 BOS	BOS	0.6111	0.5000	0.0649	3.60
2 LAC	LAC	0.6096	0.5122	0.0576	3.60
3 SAS	SAS	0.6024	0.4444	0.1518	3.23
4 MIL	MIL	0.5982	0.4024	0.1397	3.36
5 UTA	UTA	0.5948	0.4878	0.0568	4.45
6 DEN	DEN	0.5585	0.4024	0.0972	1.71
7 IND	IND	0.5511	0.4583	0.0579	1.34
8 POR	POR	0.5469	0.4730	0.0642	1.27
9 MIA	MIA	0.5426	0.4756	0.0489	0.99

E3.2 ¿Quién domina en diferencial de puntos y no solo en victorias?

The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query is as follows:

```
ROUND(AVG(a.pct), 4) AS pct_promedio,
RANK() OVER (ORDER BY AVG(a.pct) DESC) AS rank_victorias,
RANK() OVER (ORDER BY AVG(a.pct) DESC)
- RANK() OVER (ORDER BY AVG(a.margen) DESC) AS brecha
FROM analysis_team_season a
JOIN team t ON t.team_id = a.team_id
WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020
GROUP BY t.team_id, t.abbreviation
ORDER BY margen_promedio DESC
LIMIT 12;
```

The 'Data Output' tab shows the results of the query, displaying 12 rows of data. The columns are: equipo, margen_promedio, rank_margen, pct_promedio, rank_victorias, and brecha.

equipo	margen_promedio	rank_margen	pct_promedio	rank_victorias	brecha
1 TOR	4.79	1	0.6382	1	0
2 GSW	4.49	2	0.6331	2	0
3 UTA	4.45	3	0.5948	7	4
4 BOS	3.60	4	0.6111	3	-1
5 LAC	3.60	5	0.6096	4	-1
6 MIL	3.36	6	0.5982	6	0
7 SAS	3.23	7	0.6024	5	-2
8 HOU	2.32	8	0.5762	8	0
9 DEN	1.71	9	0.5585	9	0
10 IND	1.34	10	0.5511	11	1
11 POR	1.27	11	0.5469	12	1
12 OKC	1.05	12	0.5572	10	-2

E3.3 ¿Qué equipos ganan más de local que de visitante?

The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query is as follows:

```
115 FROM game
116 )
117 SELECT t.abbreviation AS equipo,
118        ROUND(AVG(win) FILTER (WHERE condicion = 'local'), 4) AS pct_local,
119        ROUND(AVG(win) FILTER (WHERE condicion = 'visita'), 4) AS pct_visita,
120        ROUND(AVG(win) FILTER (WHERE condicion = 'local')
121              - AVG(win) FILTER (WHERE condicion = 'visita'), 4) AS ventaja_local
122 FROM local_visita lv
123 JOIN team t ON t.team_id = lv.team_id
124 WHERE LEFT(season_id, 4)::int BETWEEN 2015 AND 2020
125 GROUP BY t.team_id, t.abbreviation
126 ORDER BY ventaja_local DESC
127 LIMIT 10;
```

The 'Data Output' tab shows the results of the query, displaying 10 rows. The columns are: equipo, pct_local, pct_visita, and ventaja_local.

equipo	pct_local	pct_visita	ventaja_local
PHI	0.6170	0.3697	0.2473
SAS	0.7222	0.5043	0.2179
DET	0.5365	0.3276	0.2089
DEN	0.6596	0.4635	0.1961
POR	0.6509	0.4619	0.1890
UTA	0.6957	0.5128	0.1828
MIL	0.6809	0.5084	0.1724
WAS	0.5593	0.3898	0.1695
CLE	0.5508	0.3842	0.1666
CHA	0.5455	0.3803	0.1651

Total rows: 10 Query complete 00:00:00.083 CRLF Ln 109, Col 1

E3.4 ¿Cuántas victorias compra cada millón de dólares?

The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query is as follows:

```
142 SELECT t.abbreviation AS equipo,
143        t.full_name,
144        a.victorias,
145        ROUND(ts.total_salary / 1000000.0, 1) AS nomina_millones,
146        ROUND(a.victorias / NULLIF(ts.total_salary / 1000000.0, 0), 3)
147        AS victorias_por_millon
148 FROM analysis_team_season a
149 JOIN team_salary ts ON ts.team_id = a.team_id AND ts.season_id = a.season_id
150 JOIN team t ON t.team_id = a.team_id
151 WHERE a.season_id = '2020-21'
152 ORDER BY victorias_por_millon DESC;
```

The 'Data Output' tab shows the results of the query, displaying 11 rows. The columns are: equipo, full_name, victorias, nomina_millones, and victorias_por_millon.

equipo	full_name	victorias	nomina_millones	victorias_por_millon
NYK	NYK	41	97.6	0.420
PHX	PHX	47	128.9	0.365
ATL	ATL	41	118.6	0.346
UTA	UTA	47	136.0	0.345
DEN	DEN	44	129.2	0.341
MIL	MIL	46	135.3	0.340
PHI	PHI	49	147.4	0.332
LAC	LAC	44	138.6	0.317
DAL	DAL	40	127.5	0.314
CHA	CHA	33	108.2	0.305
MIA	MIA	40	134.3	0.298

Total rows: 30 Query complete 00:00:00.067 CRLF Ln 142, Col 1

E3.5 ¿Qué equipos tienen más talento del que su nómina sugiere?

The screenshot shows pgAdmin 4 with a SQL query in the 'Query' tab. The query is as follows:

```
167 WITH gasto AS (  
168     SELECT ts.team_id,  
169            ts.total_salary,  
170            RANK() OVER (ORDER BY ts.total_salary DESC) AS rank_gasto  
171     FROM team_salary ts  
172     WHERE ts.season_id = '2020-21'  
173 ),  
174 talento AS (  
175     SELECT ps.team_id,  
176            MAX(p.pie) AS pie_estrella,  
177
```

The 'Data Output' tab shows the results of the query, displaying 12 rows of data. The columns are: equipo, nomina_millones, rank_gasto, pie_estrella, rank_talento, and brecha_a_favor.

equipo	nomina_millones	rank_gasto	pie_estrella	rank_talento	brecha_a_favor
DAL	127.5	23	0.1870	6	17
NYK	97.6	29	0.1640	13	16
DEN	129.2	17	0.2050	2	15
ATL	118.6	25	0.1610	14	11
CHI	129.0	18	0.1720	8	10
OKC	91.6	30	0.1460	21	9
MIL	135.3	7	0.1960	3	4
MIA	134.3	8	0.1900	5	3
PHI	147.4	3	0.2110	1	2
CLE	128.6	20	0.1530	18	2
POR	131.4	13	0.1650	12	1
LAL	138.3	5	0.1940	4	1

E3.6 ¿Qué equipos tienen margen financiero para reforzarse?

The screenshot shows pgAdmin 4 with a SQL query in the 'Query' tab. The query is as follows:

```
205 SELECT t.abbreviation AS equipo,  
206        ROUND(MAX(ts.total_salary) FILTER (WHERE ts.season_id = '2020-21'))  
207        / 1000000.0, 1) AS nomina_2020_21,  
208        ROUND(MAX(ts.total_salary) FILTER (WHERE ts.season_id = '2021-22'))  
209        / 1000000.0, 1) AS comprometido_2021_22,  
210        ROUND(100 * MAX(ts.total_salary) FILTER (WHERE ts.season_id = '2021-22'))  
211        / NULLIF(MAX(ts.total_salary) FILTER (WHERE ts.season_id = '2020-21'), 0), 1)  
212        AS pct_comprometido  
213 FROM team_salary ts  
214 JOIN team t ON t.team_id = ts.team_id
```

The 'Data Output' tab shows the results of the query, displaying 12 rows of data. The columns are: equipo, nomina_2020_21, comprometido_2021_22, and pct_comprometido.

equipo	nomina_2020_21	comprometido_2021_22	pct_comprometido
SAS	127.9	58.2	45.5
NYK	97.6	48.0	49.2
OKC	91.6	49.8	54.4
MIA	134.3	77.8	58.0
MEM	131.6	87.0	66.2
DAL	127.5	85.0	66.6
NOP	133.7	91.4	68.4
TOR	128.5	88.8	69.1
HOU	132.8	92.8	69.9
CLE	128.6	90.7	70.5
CHI	129.0	96.3	74.7
CHA	108.2	83.1	76.8

E3.7 ¿Qué equipos vienen en trayectoria ascendente?

The screenshot shows pgAdmin 4 with a SQL query executed against the nba_project database. The query calculates the slope of points per game over time for each team. The results table shows 12 teams with their abbreviations, full names, and calculated metrics.

```
SELECT t.abbreviation AS equipo,
       t.full_name,
       ROUND(REG_SLOPE(a.pct, LEFT(a.season_id, 4)::int)::numeric, 4)
       AS pendiente_anual,
       ROUND(AVG(a.pct), 4) AS pct_promedio,
       ROUND(MAX(a.pct) FILTER (WHERE a.season_id = '2020-21'), 4) AS pct_2020_21
FROM analysis_team_season a
JOIN team t ON t.team_id = a.team_id
WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020
GROUP BY t.team_id, t.abbreviation, t.full_name
```

equipo	full_name	pendiente_anual	pct_promedio	pct_2020_21
PHI	PHI	0.1007	0.4982	0.6806
LAL	LAL	0.0841	0.4461	0.5417
BKN	BKN	0.0823	0.4154	0.6528
PHX	PHX	0.0673	0.3633	0.6528
MIL	MIL	0.0612	0.5982	0.6389
DEN	DEN	0.0448	0.5585	0.6111
DAL	DAL	0.0240	0.4564	0.5556
UTA	UTA	0.0233	0.5948	0.6528
NYK	NYK	0.0163	0.3695	0.5694
NOP	NOP	0.0042	0.4359	0.4306
MIA	MIA	0.0028	0.5426	0.5556
LAC	LAC	0.0021	0.6096	0.6111

E3.8 ¿Qué tan bueno es el roster actual? (usa la ingesta del API)

The screenshot shows pgAdmin 4 with a SQL query executed against the nba_project database. The query calculates various roster metrics for each team. The results table shows 12 teams with their abbreviations, full names, and calculated metrics.

```
SELECT t.abbreviation AS equipo,
       COUNT(*) FILTER (WHERE s.points_per_game >= 15) AS anotadores_15plus,
       COUNT(*) FILTER (WHERE s.games_played >= 40) AS jugadores_rotacion,
       ROUND(MAX(s.points_per_game), 2) AS mejor_anotador_ppg,
       ROUND(AVG(s.points_per_game) FILTER (WHERE s.games_played >= 40), 2)
       AS ppg_promedio_rotacion
FROM player_season_stat s
JOIN team t ON t.team_id = s.team_id
WHERE s.season_id = '2020-21'
GROUP BY t.team_id, t.abbreviation
```

equipo	anotadores_15plus	jugadores_rotacion	mejor_anotador_ppg	ppg_promedio_rotacion
ATL	5	11	25.30	11.85
TOR	5	13	21.40	10.98
HOU	5	7	21.00	13.09
MIL	4	11	28.10	11.85
UTA	4	11	26.40	11.70
MIN	4	12	24.80	10.57
MIA	4	9	21.50	12.29
IND	4	10	21.20	12.68
GSW	3	11	32.00	11.24
POR	3	9	28.80	13.64
DAL	3	12	27.70	11.18
CHI	3	12	27.40	11.58

E3.9 ¿Qué equipos tienen mejor producción combinada de su núcleo?

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure for 'nba_project'. The main pane shows a SQL query in the 'Query' tab, which is a window function query to rank players by their combined core production (points, assists, and rebounds) for the 2020-21 season. The 'Data Output' pane shows the results of the query, displaying the team abbreviation, the number of players considered, and the average and median core production for each team.

```
WITH ranking_jugadores AS (
  SELECT s.team_id,
         s.player_id,
         s.points_per_game + s.assists_per_game + s.rebounds_per_game
           AS aporte_total,
         ROW_NUMBER() OVER (PARTITION BY s.team_id
                             ORDER BY s.points_per_game DESC) AS posicion
  FROM player_season_stat s
  WHERE s.season_id = '2020-21' AND s.games_played >= 30
)
SELECT equipo,
       jugadores_considerados,
       aporte_nucleo,
       aporte_promedio
FROM ranking_jugadores
GROUP BY equipo;
```

equipo	jugadores_considerados	aporte_nucleo	aporte_promedio
BKN	5	156.7	31.34
LAL	5	147.7	29.54
DEN	5	144.1	28.82
MIL	5	142.7	28.54
CHI	5	142.6	28.52
MIA	5	141.7	28.34
IND	5	139.7	27.94
UTA	5	138.5	27.70
POR	5	137.7	27.54
MIN	5	137.4	27.48
ATL	5	137.3	27.46
PHI	5	136.1	27.22

E3.10 ¿Qué equipos han sabido construir mediante el draft?

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure for 'nba_project'. The main pane shows a SQL query in the 'Query' tab, which is a join query to find teams that have drafted players who are productive (high points per game) and have a high pick number (indicating they were drafted later). The 'Data Output' pane shows the results of the query, displaying the team abbreviation, the number of productive players drafted, the average pick number, and the average points per game for those players.

```
SELECT t.abbreviation AS equipo,
       COUNT(DISTINCT d.player_id) AS picks_productivos,
       ROUND(AVG(s.points_per_game), 2) AS ppg_promedio,
       ROUND(MAX(s.pick_number), 2) AS mejor_ppg
FROM draft_selection d
JOIN team t ON t.team_id = d.team_id
JOIN player_season_stat s
  ON s.player_id = d.player_id AND s.season_id = '2020-21'
WHERE d.draft_year BETWEEN 2016 AND 2020
AND d.round_number < 5;
```

equipo	picks_productivos	ppg_promedio	mejor_ppg
PHI	7	11.11	14.30
DEN	4	21.55	26.40
ATL	4	17.10	27.70
BOS	4	17.08	26.40
CHA	4	16.00	23.70
SAS	4	13.78	15.70
MIN	4	12.68	19.30
BKN	4	12.23	12.90
UTA	4	10.75	13.70
LAL	3	17.80	23.80
SAC	3	17.43	25.20
CLE	3	17.10	24.30

E3.11 ¿Qué equipos dependen demasiado de un solo jugador?

The screenshot shows the pgAdmin 4 interface. The left pane displays the Object Explorer with the 'nba_project' database selected. The main pane shows a SQL query in the 'Query' tab, which is executed against the 'nba_project/postgres@PostgreSQL 18' database. The query is as follows:

```
WITH produccion AS (  
  SELECT s.team_id,  
         s.points_per_game,  
         MAX(s.points_per_game) OVER (PARTITION BY s.team_id) AS mejor  
  FROM player_season_stat s  
  WHERE s.season_id = '2020-21' AND s.games_played >= 30  
)  
SELECT t.abbreviation AS equipo,  
       ROUND(MAX(p.mejor), 2) AS mejor_anotador_ppg,  
       ROUND(SUM(p.points_per_game), 2) AS puntos_plantel,  
       ROUND(SUM(p.mejor - MAX(p.mejor), 2) AS pct_dependencia
```

The Data Output pane shows the results of the query, displaying 12 rows of data. The columns are: equipo, mejor_anotador_ppg, puntos_plantel, and pct_dependencia.

equipo	mejor_anotador_ppg	puntos_plantel	pct_dependencia
1 ORL	15.60	113.80	13.71
2 MIA	21.50	154.90	13.88
3 CHA	20.40	144.10	14.16
4 TOR	21.40	142.80	14.99
5 IND	21.20	141.40	14.99
6 SAS	21.60	138.50	15.60
7 MEM	19.10	122.20	15.63
8 LAL	25.00	155.90	16.04
9 MIN	24.80	151.70	16.35
10 DEN	26.40	160.40	16.46
11 OKC	23.70	141.70	16.73
12 LAC	24.80	147.70	16.79

E3.12 - ÍNDICE COMPUESTO DE INVERSIÓN

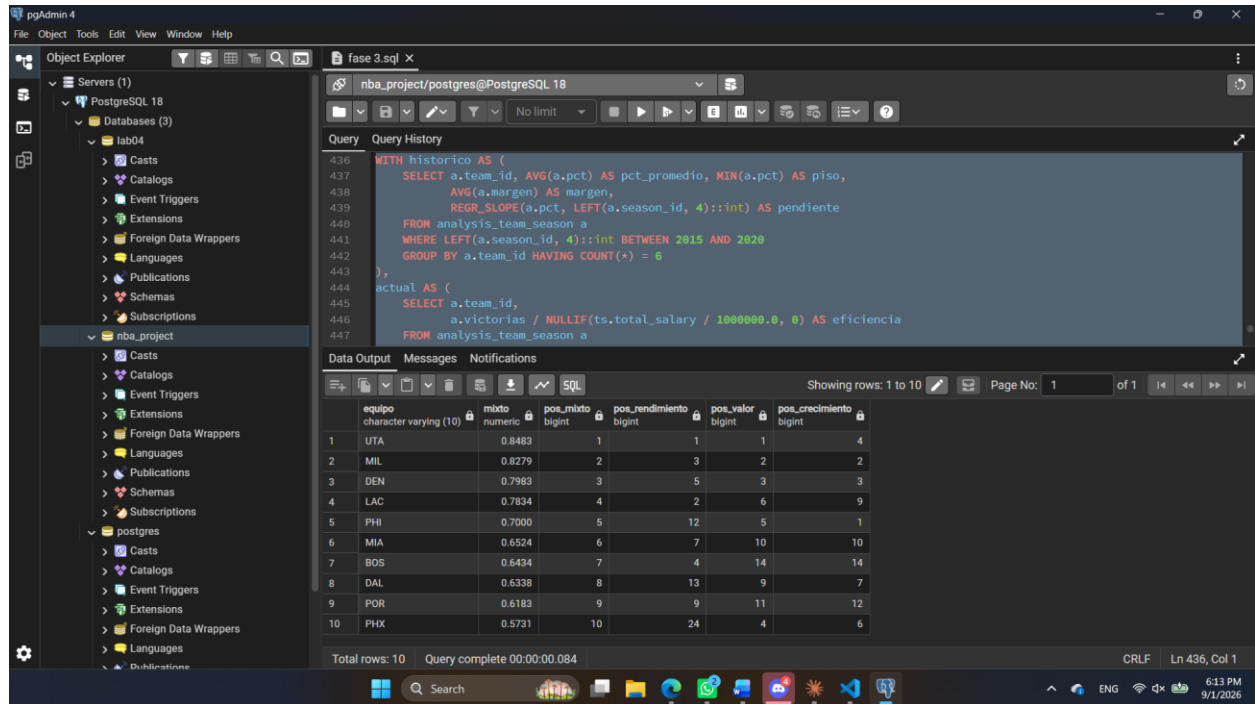
The screenshot shows the pgAdmin 4 interface. The left pane displays the Object Explorer with the 'nba_project' database selected. The main pane shows a SQL query in the 'Query' tab, which is executed against the 'nba_project/postgres@PostgreSQL 18' database. The query is as follows:

```
SELECT a.team_id,  
       AVG(a.pct) AS pct_promedio,  
       MIN(a.pct) AS piso,  
       AVG(a.margen) AS margen,  
       REGR_SLOPE(a.pct, LEFT(a.season_id, 4)::int) AS pendiente  
FROM analysis_team_season a  
WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020  
GROUP BY a.team_id  
HAVING COUNT(*) = 6
```

The Data Output pane shows the results of the query, displaying 30 rows of data. The columns are: equipo, full_name, pct_ses_temporadas, margen_promedio, victorias_2020_21, nomina_millones, victorias_por_millon, tendencia, and indice_invercion.

equipo	full_name	pct_ses_temporadas	margen_promedio	victorias_2020_21	nomina_millones	victorias_por_millon	tendencia	indice_invercion
1 UTA	UTA	0.595	4.45	47	136.0	0.345	0.0233	
2 MIL	MIL	0.598	3.36	46	135.3	0.340	0.0612	
3 DEN	DEN	0.559	1.71	44	129.2	0.341	0.0448	
4 LAC	LAC	0.610	3.60	44	138.6	0.317	0.0021	
5 PHI	PHI	0.498	-0.13	49	147.4	0.332	0.1007	
6 MIA	MIA	0.543	0.99	40	134.3	0.298	0.0028	
7 BOS	BOS	0.611	3.60	36	132.0	0.273	-0.0125	
8 DAL	DAL	0.456	-0.03	40	127.5	0.314	0.0240	
9 POR	POR	0.547	1.27	38	131.4	0.289	-0.0022	
10 PHX	PHX	0.363	-4.16	47	128.9	0.365	0.0673	
11 IND	IND	0.551	1.34	33	129.4	0.255	-0.0040	
12 NYK	NYK	0.360	-3.83	41	97.6	0.420	0.0163	

E3.13 - ANÁLISIS DE SENSIBILIDAD



The screenshot displays the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure for 'PostgreSQL 18', including 'Databases (3)', 'lab04', 'Casts', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', and 'nba_project'. The 'nba_project' database is selected, showing its own structure. The main query editor shows a SQL query named 'fase 3.sql' with the following content:

```
436 WITH historico AS (  
437     SELECT a.team_id, AVG(a.pct) AS pct_promedio, MIN(a.pct) AS piso,  
438            AVG(a.margen) AS margen,  
439            REGR_SLOPE(a.pct, LEFT(a.season_id, 4)::int) AS pendiente  
440     FROM analysis_team_season a  
441     WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020  
442     GROUP BY a.team_id HAVING COUNT(*) = 6  
443 ),  
444 actual AS (  
445     SELECT a.team_id,  
446            a.victorias / NULLIF(ts.total_salary / 1000000.0, 0) AS eficiencia  
447     FROM analysis_team_season a
```

The 'Data Output' tab shows the results of the query in a table with 10 rows. The table has the following columns: 'equipo' (character varying (10)), 'mixto' (numeric), 'pos_mixto' (bigint), 'pos_rendimiento' (bigint), 'pos_valor' (bigint), and 'pos_crecimiento' (bigint). The data is as follows:

	equipo	mixto	pos_mixto	pos_rendimiento	pos_valor	pos_crecimiento
1	UTA	0.8483	1	1	1	4
2	MIL	0.8279	2	3	2	2
3	DEN	0.7983	3	5	3	3
4	LAC	0.7834	4	2	6	9
5	PHI	0.7000	5	12	5	1
6	MIA	0.6524	6	7	10	10
7	BOS	0.6434	7	4	14	14
8	DAL	0.6338	8	13	9	7
9	POR	0.6183	9	9	11	12
10	PHX	0.5731	10	24	4	6

The status bar at the bottom indicates 'Total rows: 10', 'Query complete 00:00:00.084', and 'CRLF Ln 436, Col 1'. The system clock shows 6:13 PM on 9/1/2026.